SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE NAME: Artificial Intelligence COURSE CODE: CSA1741

CO1-AT1-Analytical Problem Solving

Register Number	**192412204**
Name	**S. Arshad Ali**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks: 50**

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

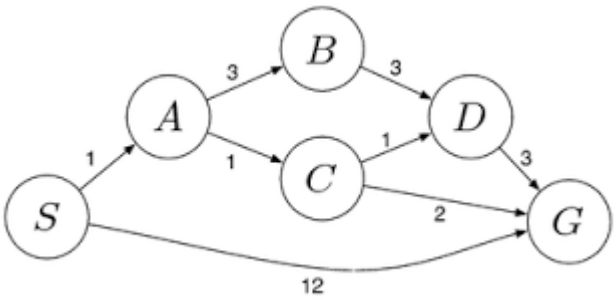
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost

path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Code of Conduct:

I _____ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

ANSWERS

1)Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Ans:

1. Aim

To obtain exactly 2 gallons of water in a 4-gallon jug using a 4-gallon jug and a 3-gallon jug without using any measuring instruments.

2. Problem Description

Two jugs are available:

- Jug A: Capacity of 4 gallons.
- Jug B: Capacity of 3 gallons.

Initially, both jugs are empty. We have a water pump to fill the jugs, and water can be poured from one jug to another or emptied onto the ground. The objective is to measure exactly 2 gallons of water in Jug A.

3. Assumptions

1. The 4-gallon jug and 3-gallon jug have no measurement markings.
2. Either jug can be filled completely using the pump.
3. Water can be emptied onto the ground.
4. Water can be transferred between the jugs until the source is empty or the destination is full.
5. No other measuring devices are available.

4. Algorithm

1. Start with both jugs empty.
2. Fill the 3-gallon jug completely.
3. Pour water from the 3-gallon jug into the 4-gallon jug.
4. Fill the 3-gallon jug again.
5. Pour water into the 4-gallon jug until it becomes full.
6. Empty the 4-gallon jug onto the ground.
7. Transfer the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug.
8. Stop when the 4-gallon jug contains exactly 2 gallons.

5. Step-by-Step Solution

Let the state be represented as \((A,B)\), where \(A\) is the amount of water in the 4-gallon jug and \(B\) is the amount in the 3-gallon jug.

Step	Action performed	Jug A (4 gal)	Jug B (3 gal)
1	Initially, both jugs are empty	0	0
2	Fill Jug B completely	0	3
3	Pour Jug B into Jug A	3	0
4	Fill Jug B again	3	3
5	Pour from Jug B into Jug A until Jug A is full	4	2
6	Empty Jug A onto the ground	0	2
7	Pour the remaining water from Jug B into Jug A	2	0

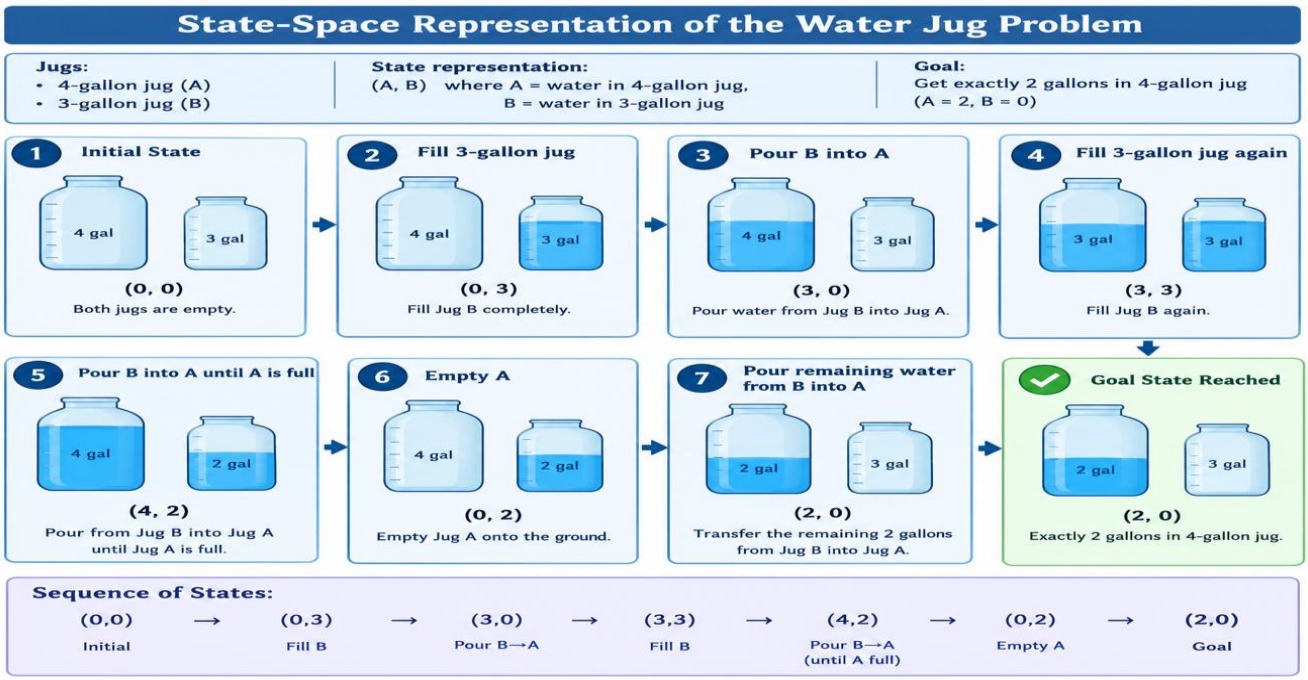
Final result: Jug A contains exactly 2 gallons of water, and Jug B is empty.

6. State-Space Representation

Initial State

\((0,0)\) — Both jugs are empty

Fill Jug B
\((0,3)\)
Transfer water to Jug A
\((3,0)\)
Fill Jug B and fill Jug A
\((4,2)\)
Empty Jug A
\((0,2)\)
Goal State Reached
\((2,0)\) — Exactly 2 gallons in Jug A



7. Result

Thus, the water jug problem is solved successfully by applying a sequence of filling, pouring, and emptying operations. Exactly 2 gallons of water are obtained in the 4-gallon jug without using any external measuring device. This problem demonstrates state-space search and problem-solving techniques used in Artificial Intelligence.

2) Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

I. **What are the percept's for this agent?**
II. **Characterize the operating environment.**
III. **What are the actions the agent can take?**
IV. **How can one evaluate the performance of the agent?**
V. **What sort of agent architecture do you think is most suitable for this agent? Why?**

ANS:

Introduction

A Mars Rover is an autonomous robotic vehicle designed to explore the surface of Mars. It collects soil and rock samples, analyzes their composition, captures images, and sends scientific data to Earth. In Artificial Intelligence (AI), a Mars Rover is an example of an intelligent agent that perceives its environment through sensors and takes actions to achieve its goals.

I. What are the percepts for this agent?

Percepts are the information received by an intelligent agent through its sensors.
The Mars Rover receives the following percepts:

1. Camera images: Images of rocks, soil, hills, and surrounding terrain.
2. Distance measurements: Information about nearby obstacles using appropriate sensors.
3. Chemical composition: Data about soil and rock samples collected by scientific instruments.
4. Temperature: Environmental and instrument temperature readings.
5. Position and orientation: Information about the rover's location, direction, and movement.
6. Power level: Battery charge and available energy.
7. Communication status: Information about its connection with Earth or orbiting relay spacecraft.

II. Characterize the operating environment.

The operating environment of a Mars Rover can be characterized as follows:

Property	Characterization	Explanation
Observability	Partially observable	The rover cannot directly observe every part of Mars.
Determinism	Stochastic	Wheel slippage, dust, and unexpected terrain can make outcomes uncertain.
Episodic or sequential	Sequential	Each action can affect future actions and decisions.
Static or dynamic	Dynamic	Environmental conditions can change over time.
Discrete or continuous	Hybrid	Actions and decisions may be discrete, while position and time are continuous.
Single-agent or multi-agent	Primarily single-agent	The rover acts independently, although it communicates with Earth-based operators and other spacecraft.

III. What are the actions the agent can take?

The Mars Rover can perform the following actions:

1. Move: Drive forward, backward, or turn to explore the surface.
2. Avoid obstacles: Change its route to avoid rocks, craters, and dangerous terrain.
3. Capture images: Take photographs of the surroundings for analysis.
4. Collect samples: Use its robotic arm and drilling equipment to gather soil and rock samples.
5. Analyze samples: Use scientific instruments to study the chemical and mineral composition of samples.
6. Transmit data: Send images, scientific results, and status information to Earth.
7. Manage power: Reduce energy consumption and enter a low-power state when necessary.
8. Plan routes: Select suitable paths to reach scientific targets safely.

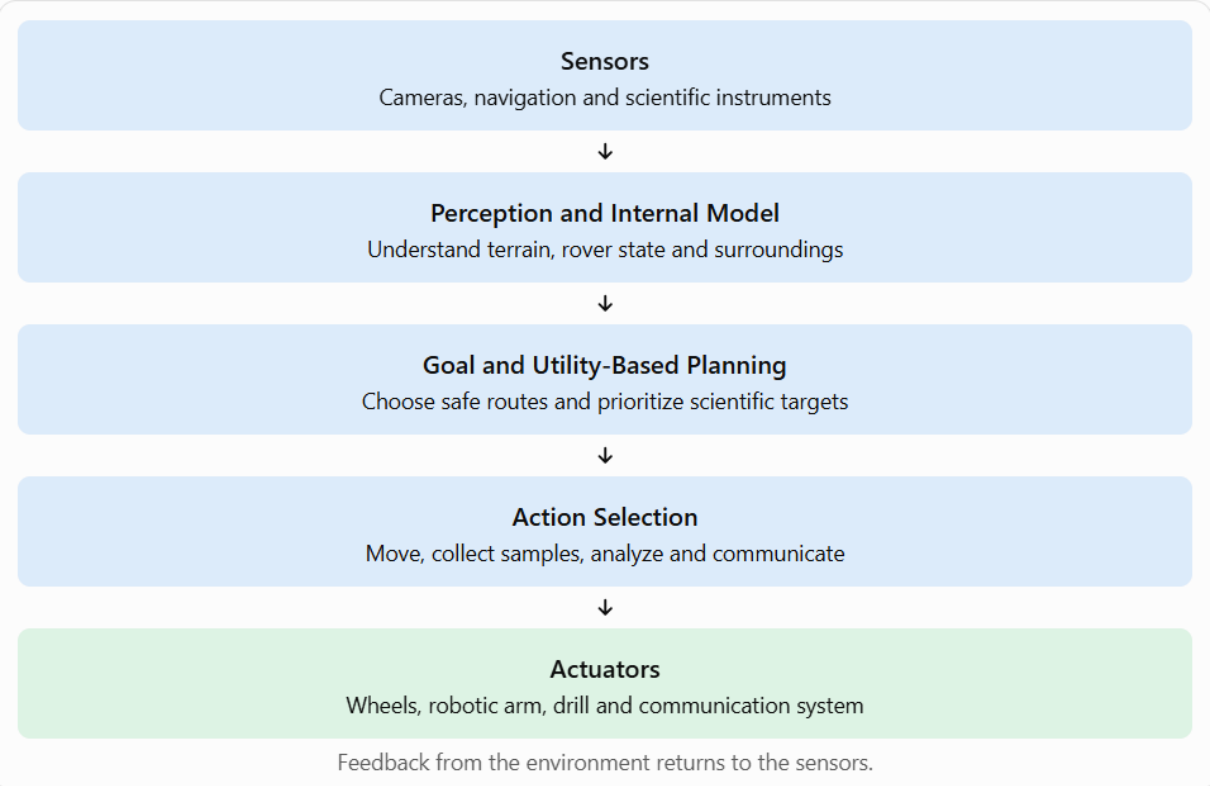
IV. How can one evaluate the performance of the agent?

The performance of a Mars Rover can be evaluated using the following criteria:

- Scientific success: Number and quality of useful samples collected and analyzed.
- Navigation accuracy: Ability to reach the required locations.
- Safety: Avoidance of obstacles and prevention of damage.
- Energy efficiency: Effective use of limited battery power.
- Time efficiency: Completion of tasks within the available mission time.
- Communication reliability: Successful transmission of scientific data to Earth.

- Mission achievement: Achievement of the assigned exploration objectives.

A rover performs well when it collects valuable scientific information, navigates safely, uses resources efficiently, and completes its mission objectives.



Reasons for choosing this architecture:

1. Model-based: Maintains an internal representation of the rover's surroundings, including previously observed obstacles.
2. Goal-based: Plans actions to achieve objectives, such as reaching a rock formation or collecting a sample.
3. Utility-based: Compares alternatives based on safety, scientific value, time, and energy consumption.
4. Learning capability: Learning components can improve terrain classification and decision-making using experience and collected data.

This hybrid architecture allows the rover to make intelligent decisions even when communication with Earth is delayed or the environment is uncertain.

Conclusion

The Mars Rover is an intelligent agent that uses sensors to perceive Mars, processes information to make decisions, and performs actions to explore the planet. A hybrid model-based, goal-based, and utility-based architecture is suitable because it supports autonomous navigation, scientific investigation, efficient resource management, and safe mission execution.

3) Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

ANS:

1. Introduction

The 8-Queens problem is a classical Artificial Intelligence search problem. The objective is to place 8 queens on an \(8 \times 8\) chessboard such that no two queens attack each other.
A queen can attack another queen if they are in the same row, column, or diagonal. Therefore, the solution must ensure that all 8 queens are placed without any attacks.

2. Problem Formulation

The main components of the 8-Queens problem are:

Component	Description
Initial state	An empty \(8 \times 8\) chessboard.
State space	All possible configurations of queens on the board.
Actions	Place a queen in a valid square, usually one queen per column.
Transition model	Adds a queen to the selected square and produces a new board configuration.
Goal test	Checks whether 8 queens are placed and no two queens attack each other.
Path cost	Each queen placement can have a cost of 1; a complete solution requires 8 placements.

Constraints

For any two queens at positions \((r_i,c_i)\) and \((r_j,c_j)\), they must satisfy:

1. Different rows: \(r_i \ne r_j\)
2. Different columns: \(c_i \ne c_j\)
3. Different diagonals: \(|r_i-r_j| \ne |c_i-c_j|\)

If any of these conditions is violated, the configuration is invalid.

3. Exploring the Search Space

The search space can be explored by placing queens one column at a time. If a queen attacks another queen, that configuration is rejected, and the search tries another position.



4. Search Strategies

A. Breadth-First Search (BFS)

- Explores all configurations at one level before moving to the next level.
- Guarantees a solution if one exists and the search space is finite.
- Requires large amounts of memory because it stores many configurations.

B. Depth-First Search (DFS)

- Places queens one by one and explores one branch deeply before trying another.
- Uses less memory than BFS.
- Can waste time exploring invalid configurations unless constraints are checked.

C. Backtracking Search

- Places one queen in each column.
- Checks whether the new position conflicts with previously placed queens.
- If a conflict occurs, removes the queen and tries another row.
- Efficiently avoids exploring many invalid configurations.

Backtracking is a particularly suitable strategy for the 8-Queens problem because it rejects invalid partial configurations early.

5. Example of a Valid Solution

One valid arrangement, represented by the row number chosen for each column, is:
\[[1,5,8,6,3,7,2,4] \]
This means:

- Column 1 → Row 1
- Column 2 → Row 5
- Column 3 → Row 8
- Column 4 → Row 6
- Column 5 → Row 3
- Column 6 → Row 7
- Column 7 → Row 2
- Column 8 → Row 4

All queens occupy different rows and columns, and no two queens share a diagonal.

6. Conclusion

The 8-Queens problem demonstrates how Artificial Intelligence uses state-space search to solve constraint-satisfaction problems. By formulating the initial state, actions, transition model, constraints, and goal test, the problem can be solved using DFS and backtracking. Backtracking is effective because it eliminates invalid configurations early and finds a valid arrangement without checking every possible board configuration.

4) A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

ANS:

1. Introduction

OLA is a cab-booking mobile application that helps customers travel from a source location to a destination. Customers can choose different cab types, such as Mini, Micro, Sedan, Shared, and Prime, based on their comfort, budget, and travel requirements.
In Artificial Intelligence, this can be modeled using a goal-based problem-solving agent. The agent identifies the customer's destination as the goal, considers available cab options, and selects a suitable cab to complete the journey.

2. Type of Problem-Solving Agent

Goal-based problem-solving agent is suitable for this problem.
The agent works through the following stages:

1. Goal formulation: Identify the customer's pickup and destination locations.

2. Problem formulation: Identify available cabs, estimated fares, travel time, and customer preferences.
3. Search and selection: Compare available cab types and choose a suitable option.
4. Execution: Book the selected cab and provide journey details.
5. Evaluation: Check whether the customer reaches the destination successfully.

A utility-based component can also be included to compare price, comfort, waiting time, and travel duration.

3. Problem Formulation

Component	**Description**
Initial state	Customer opens the OLA application.
State space	Available cab types, routes, and booking options.
Initial input	Pickup location and destination.
Actions	Select cab, compare fares, choose route, and book cab.
Goal state	Customer reaches the destination successfully.
Path cost	Fare, travel time, waiting time, and other travel costs.

5. Pseudocode

```
ALGORITHM OLA_CAB_BOOKING

BEGIN
  OPEN OLA application

  INPUT pickup_location
  INPUT destination

  IF pickup_location = destination THEN
    DISPLAY "Pickup and destination must be different"
    STOP
  END IF

  available_cabs ← GET_AVAILABLE_CABS(
                    pickup_location,
                    destination
                  )

  IF available_cabs is EMPTY THEN
    DISPLAY "No cabs available"
    STOP
  END IF

  DISPLAY available_cabs
  DISPLAY estimated_fare and arrival_time

  INPUT customer_preference
  // Mini, Micro, Sedan, Shared, Prime, etc.

  suitable_cabs ← FILTER available_cabs
                  USING customer_preference

  selected_cab ← SELECT_BEST_CAB(
                   suitable_cabs,
                   fare,
                   comfort,
```

```
            arrival_time
        )

    DISPLAY selected_cab details

    INPUT customer_confirmation

    IF customer_confirmation = YES THEN
        booking ← BOOK_CAB(selected_cab)

        DISPLAY booking confirmation
        TRACK cab_location

        WHILE destination NOT reached DO
            UPDATE cab_location
        END WHILE

        DISPLAY "Destination reached successfully"
    ELSE
        DISPLAY "Booking cancelled"
    END IF
END
```

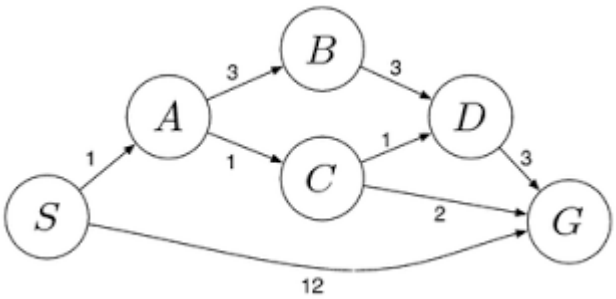
6. Advantages

1. Helps customers choose a suitable cab according to their preferences.
2. Compares fares and estimated arrival times.
3. Reduces the time needed to find transportation.
4. Supports decision-making using cost, comfort, and availability.
5. Can adapt to changing cab availability and traffic conditions.

7. Conclusion

The OLA cab-booking application can be modeled as a goal-based problem-solving agent, with utility-based decision-making to select the most suitable cab. It takes the pickup and destination as inputs, searches available options, selects a cab according to customer preferences, and completes the booking process to help the customer reach the destination.

5) A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm. The delivery network is represented as a weighted graph:



ANS:

1. Given Data

The weighted graph represents a delivery network in which each edge has a travel cost.

- Initial node: \(S\)

- Goal node: \(G\)
- Search algorithm: Uniform Cost Search (UCS)
- Objective: Find the path from \(S\) to \(G\) with minimum total cost.

Edges and costs

Edge	Cost
\(S \rightarrow A\)	1
\(S \rightarrow G\)	12
\(A \rightarrow B\)	3
\(A \rightarrow C\)	1
\(B \rightarrow D\)	3
\(C \rightarrow D\)	1
\(C \rightarrow G\)	2
\(D \rightarrow G\)	3

2. UCS Algorithm

Uniform Cost Search expands the node with the lowest cumulative path cost first. It uses a priority queue ordered by total cost from the starting node.

The cumulative cost is calculated as:

\[g(n)=g(\text{parent})+\text{edge cost} \]

Unlike Breadth-First Search, UCS considers the costs of the edges, not just the number of steps.

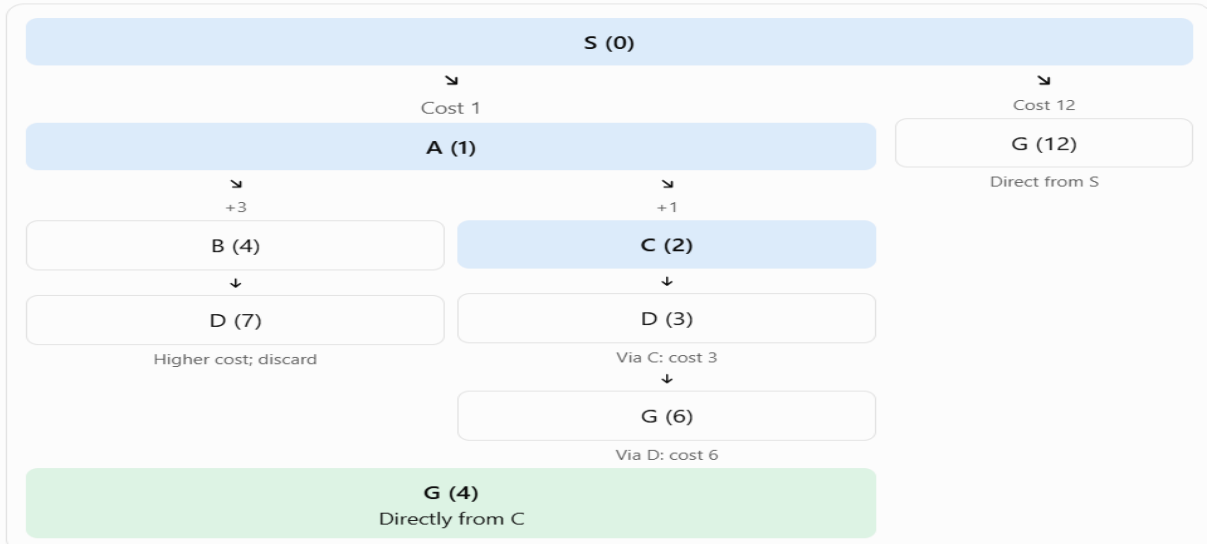
3. Step-by-Step Execution of UCS

Initially, the priority queue contains \(S(0)\), where the number represents the cumulative cost.

Step	Expanded node	Successors generated	Priority queue (lowest cost first)
1	\(S(0)\)	\(A(1), G(12)\)	\(A(1), G(12)\)
2	\(A(1)\)	\(B(4), C(2)\)	\(C(2), B(4), G(12)\)
3	\(C(2)\)	\(D(3), G(4)\)	\(D(3), B(4), G(4)\)
4	\(D(3)\)	\(G(6)\)	\(B(4), G(4), G(6)\)
5	\(B(4)\)	\(D(7)\)	\(G(4), G(6)\)
6	\(G(4)\)	Goal reached	Stop

Note: The table uses a tie-breaking rule that expands \(B(4)\) before \(G(4)\). Since \(D(3)\) has already been reached more cheaply than \(D(7)\), the latter is discarded. UCS terminates when the lowest-cost goal node is removed from the priority queue.

4. Search Tree



5. Calculate the Path Costs
Possible paths from \(S\) to \(G\) include:
Path 1: \(S →G\)
\[C_1=12 \]
Path 2: \(S →A → C → G\)
\[C_2=1+1+2=\boxed{4} \]
Path 3: \(S → A → C \r→ D \→G\)
\[C_3=1+1+1+3=6 \]
Path 4: \(S \→A \→B \→D \→G\)
\[C_4=1+3+3+3=10 \]
Conclusion: Uniform Cost Search finds the least-cost path S→A→C→Gwith a total cost of 4. UCS guarantees an optimal solution when edge costs are non-negative, as they are in this graph.